

1. Logika cyfrowa

| Układ              | Formuła / sygnatura                                                 | Opis                                                                                                                  |
|--------------------|---------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------|
| Sumatory           |                                                                     |                                                                                                                       |
| Półsumator         | $s = x \oplus y$<br>$c = x \& y$                                    | Dodaje dwa bity. <b>s</b> to suma, <b>c</b> to bit przeniesienia (carry).                                             |
| Pełny sumator (FA) | $s = x \oplus y \oplus ci$<br>$co = x \& y \mid ci \& (x \oplus y)$ | Jak półsumator, ale przyjmuje też przeniesienie wejściowe <b>ci</b> .                                                 |
| Sumator n-bitowy   | $n \times \text{FA}$                                                | Kaskada (ripple-carry): wyjście $C_i$ trafia na wejście kolejnego FA. Ostatnie <b>co</b> to <b>Carry Out</b> całości. |

|                     |                         |                                                                                                                          |
|---------------------|-------------------------|--------------------------------------------------------------------------------------------------------------------------|
| Układy kombinacyjne |                         |                                                                                                                          |
| Dekoder             | $n \rightarrow 2^n$     | Wejście koduje liczbę $k$ . Na wyjściu zapalony jest <b>tylko</b> bit nr $k$ .                                           |
| Multiplekser        | $2^n + n \rightarrow 1$ | $2^n$ bitów danych + $n$ bitów sterujących $S$ . Na wyjście przechodzi $S$ -ty bit danych.                               |
| ALU                 | $A, B, f \rightarrow C$ | Dekoder na bitach $f$ wybiera operację (np. $A + B$ , $A \mid B$ , $A \& B$ ); multipleksowanie wyników bramkami AND/OR. |

|                             |                                  |                                                                                                                                                    |
|-----------------------------|----------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------|
| Układy sekwencyjne (pamięć) |                                  |                                                                                                                                                    |
| Przerzutnik S-R             | <b>S</b> - set, <b>R</b> - reset | Przechowuje 1 bit ( $Q$ ). <b>S=1</b> ustawia $Q = 1$ , <b>R=1</b> zeruje, <b>S=R=0</b> trzyma stan. Zbudowany z dwóch sprzężonych <b>NOR</b> -ów. |
| Sterowany poziomem          | aktywny gdy <b>CLK</b> = 1       | Wejścia <b>S</b> / <b>R</b> bramkowane AND-em z zegarem. Stan zmienia się tylko przy <b>CLK=1</b> .                                                |
| Sterowany zboczem           | zbocze 1 $\rightarrow$ 0         | Dwa przerzutniki poziomowe w kaskadzie (master-slave), drugi z zanegowanym zegarem. Stan przepisuje się w <b>momencie</b> zmiany zegara.           |

2. Typy danych

| Typ    | char | short | int | long | float | double | void* |
|--------|------|-------|-----|------|-------|--------|-------|
| x86-64 | 1    | 2     | 4   | 8    | 4     | 8      | 8     |

3. Kodowanie liczb i konwersja

$$\text{B2U}(X) = \sum_{i=0}^{w-1} x_i 2^i \quad \text{B2T}(X) = -x_{w-1} 2^{w-1} + \sum_{i=0}^{w-2} x_i 2^i$$

$$\text{T2U}(x) = x < 0 ? x + 2^w : x \quad \text{U2T}(u) = u > \text{TMax} ? u - 2^w : u$$

Skróty: **B** = bity, **U** = unsigned, **T** = ze znakiem (kod U2).  
Stąd **B2U** = bity  $\rightarrow$  unsigned, podobnie **U2T**, **UMax**, **TMax**, **UAdd**, **TAdd**.

| Stała | Bity          | Wartość                          |
|-------|---------------|----------------------------------|
| UMax  | <b>11...1</b> | $2^w - 1$                        |
| TMax  | <b>01...1</b> | $2^{w-1} - 1$ ( <b>INT_MAX</b> ) |
| TMin  | <b>10...0</b> | $-2^{w-1}$ ( <b>INT_MIN</b> )    |
| −1    | <b>11...1</b> | te same bity co UMax             |

Promocja w C: **unsigned** i **int** w jednym wyrażeniu/porównaniu ( $< > ==$ )  $\rightarrow$  **int** promowany do **unsigned** (bity bez zmian,  $-1 \rightarrow \text{UMax}$ ).

4. Rozszerzanie, obcinanie i przesunięcia

|                                |                                               |
|--------------------------------|-----------------------------------------------|
| Rozszerz. 0 ( <b>zext</b> )    | <b>unsigned</b> : dopisuje 0 z góry           |
| Rozszerz. znak ( <b>sext</b> ) | <b>signed</b> : powiela bit znaku (MSB)       |
| <b>x &lt;&lt; k</b>            | $x \cdot 2^k$ (sgn./uns.)                     |
| <b>u &gt;&gt; k</b>            | $\lfloor u/2^k \rfloor$ — logiczne (zera)     |
| <b>x &gt;&gt; k</b>            | arytmetyczne (kopia MSB), zaokr. do $-\infty$ |
| $x/2^k$ do 0                   | <b>(x + (1&lt;&lt;k)-1) &gt;&gt; k</b>        |

Strength red.: **x\*24**  $\rightarrow$  **(x<<5)-(x<<3)**.

5. Operacje, overflow i endianness

|                                 |                                                    |
|---------------------------------|----------------------------------------------------|
| UAdd/TAdd                       | $(u + v) \bmod 2^w$                                |
| Nadmiar (+)                     | $u, v > 0$ , a wynik $< 0$ (wynik $-2^w$ )         |
| Nadmiar (-)                     | $u, v < 0$ , a wynik $\geq 0$ (wynik $+2^w$ )      |
| Negacja U2                      | <b>-x = ~x+1</b> ; <b>-TMin=TMin</b> , <b>-0=0</b> |
| Wykr. nadmiaru ( <b>s=x+y</b> ) | <b>((s^x)&amp;(s^y))&gt;&gt;(w-1)</b>              |
| Maska / abs                     | <b>m=x&gt;&gt;31; abs=(x^m)-m</b>                  |

| Schemat       | Najniższy adres | <b>0x0123</b>   |
|---------------|-----------------|-----------------|
| Big Endian    | MSB             | <b>[01][23]</b> |
| Little Endian | LSB             | <b>[23][01]</b> |

6. Zmiennoprzecinkowe (IEEE 754)

$$V = (-1)^s \times M \times 2^E \qquad \text{Bias} = 2^{k-1} - 1$$

| Format | bity | s | exp (k) | frac (n) |
|--------|------|---|---------|----------|
| float  | 32   | 1 | 8       | 23       |
| double | 64   | 1 | 11      | 52       |

Kategorie wartości

| Kategoria      | exp                                  | Własności                                                                                                                                                                           |
|----------------|--------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Znormalizowane | $\neq 0 \dots 0$<br>$\neq 1 \dots 1$ | $E = \text{exp} - \text{Bias}$ . $M = 1.\text{frac}$ .<br>Najczęściej ułożone wokół 0.                                                                                              |
| Zdenormaliz.   | $== 0 \dots 0$                       | $E = 1 - \text{Bias}$ . $M = 0.\text{frac}$ .<br>Reprezentują +0.0 i -0.0 (frac = 0)<br>oraz liczby bardzo bliskie zera.                                                            |
| Specjalne      | $== 1 \dots 1$                       | Gdy <b>frac == 0</b> , to $+\infty$ lub $-\infty$<br>(nadmiar/dzielenie przez 0).<br>Gdy <b>frac <math>\neq</math> 0</b> , to <b>NaN</b> (np. $\sqrt{-1}$ ,<br>$\infty - \infty$ ). |

Zaokrąglanie GRS i Round-to-Even

Domyślny tryb to **Round-to-Even** (zapobiega błędowi statycznemu przy sumowaniu).



**G** - ostatni zachowany, **R** - pierwszy usunięty, **S** - OR reszty

| Warunek         | Ułamek | Akcja                                                             |
|-----------------|--------|-------------------------------------------------------------------|
| <b>R=0</b>      | < 0.5  | W dół (odcięcie)                                                  |
| <b>R=1, S=1</b> | > 0.5  | W górę (+1 do <b>G</b> )                                          |
| <b>R=1, S=0</b> | = 0.5  | <b>Do parzystej:</b> w górę gdy <b>G=1</b> , w dół gdy <b>G=0</b> |

Własności matematyczne i rzutowanie

- **Brak łączności:**  $(a + b) + c \neq a + (b + c)$ , przez zaokrąglenia i utratę danych.
- **Brak rozdzielności:**  $a(b + c) \neq ab + ac$ .
- **Rzutowanie `int` → `float`:** Może stracić precyzję (zaokrągla zgodnie z trybem).
- **Rzutowanie `int` → `double`:** Dokładne i bezstratne (bo 52 bity mantysy > 32 bity inta).
- **Rzutowanie `float/double` → `int`:** Obcina ułamek w stronę 0. Jeśli poza zakresem lub **NaN** → z reguły zwraca **TMin** (`0x80000000`).